



UITs

Future will be better than the past

University of Information Technology & Sciences

An initiative of *PHP Family*

Microprocessors, Microcontrollers and Assembly Language Lab

Lab Report - 4

Course Title : Microprocessors, Microcontrollers and Assembly
Language Lab

Course Code : CSE0611324

Submitted to:

Teacher's Name	Designation
Tahmina Yeasmin Lima	Lecturer
Abrar Mahbub Tanim	Lecturer

Submitted by:

Student Name : Gaus Saraf Murady

Student ID : 0432410005101088

Semester : Autumn 2026

Batch : 55

Department : CSE

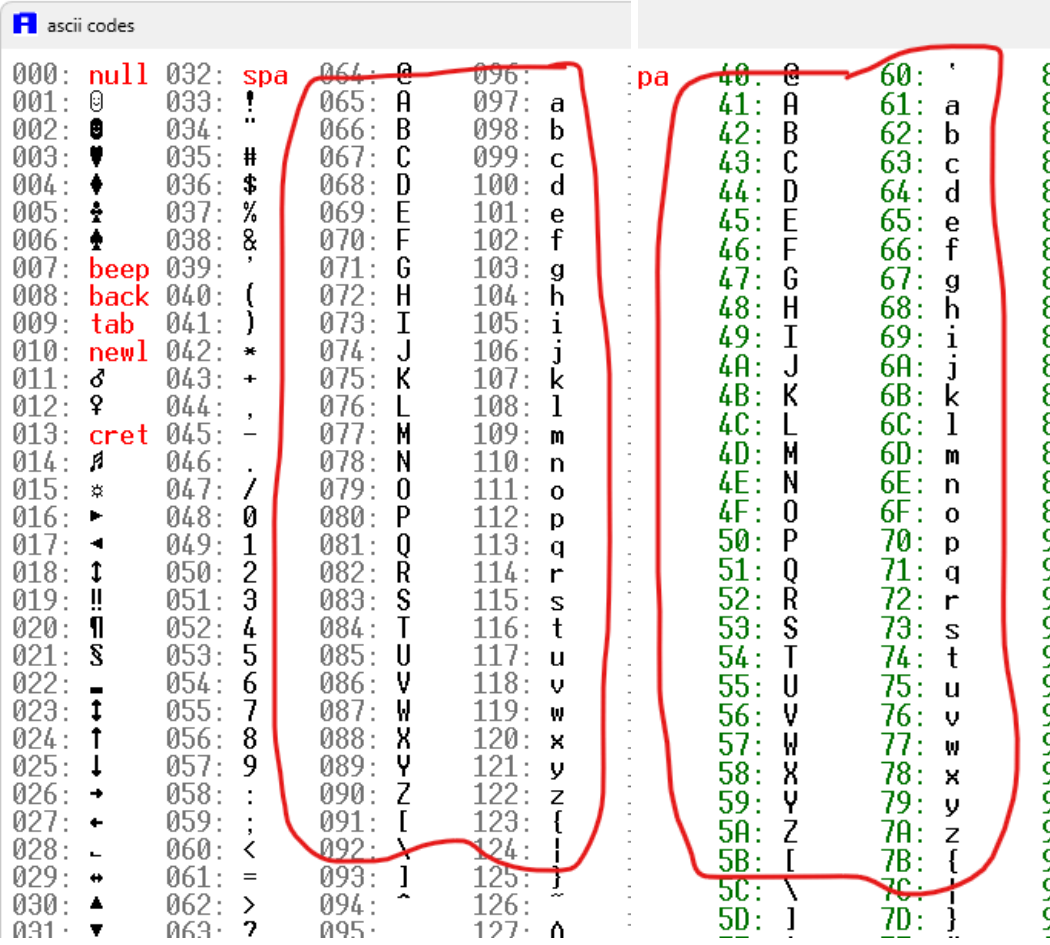
Date of Submission: 06.09.26

Experiment No : 4

Experiment Name: Conversion of lowercase letter to uppercase letter and vice versa

Introduction:

In this lab, we worked with Alphabets to convert Small Letter inputs to Capital Letter counterparts. The core logic is the conversion of ASCII values `sub al, 20h` or `sub al, 32` so the output comes out as the Capital Letter.



ascii codes			
000: null	032: spa	064: @	096: a
001: 0	033: !	065: A	097: b
002: 1	034: "	066: B	098: c
003: 2	035: #	067: C	099: d
004: 3	036: \$	068: D	100: e
005: 4	037: %	069: E	101: f
006: 5	038: &	070: F	102: g
007: beep	039: '	071: G	103: h
008: back	040: (	072: H	104: i
009: tab	041:)	073: I	105: j
010: newl	042: *	074: J	106: k
011: ♂	043: +	075: K	107: l
012: ♀	044: ,	076: L	108: m
013: cret	045: -	077: M	109: n
014: ♂	046: .	078: N	110: o
015: *	047: /	079: O	111: p
016: ►	048: 0	080: P	112: q
017: ◀	049: 1	081: Q	113: r
018: ⬆	050: 2	082: R	114: s
019: !!	051: 3	083: S	115: t
020: ¶	052: 4	084: T	116: u
021: §	053: 5	085: U	117: v
022: ▬	054: 6	086: V	118: w
023: ⬆	055: 7	087: W	119: x
024: ⬆	056: 8	088: X	120: y
025: ⬆	057: 9	089: Y	121: z
026: →	058: :	090: Z	122: {
027: +	059: ;	091: [	123: }
028: -	060: <	092: \	124: ~
029: +	061: =	093:]	125: ^
030: ▲	062: >	094: ^	126: _
031: ▼	063: ?	095: ~	127: `

For our assignment, in addition to the core logic, we had to use the reverse logic (Capital→Small conversion) with `JBE` and `CMP` commands (`If-Else` equivalent of Programming Languages).

Code:

```
.model small
.stack 100h

.data
    msg1 db "Input: $"
    msg2 db "Output : $"

    char db ?

.code
main proc

    mov ax, @data
    mov ds, ax

    ; INPUT

    mov dx, offset msg1
    mov ah, 09h
    int 21h

    mov ah, 01h
    int 21h
    ; sub al, 20h
; WILL BE ADD / SUM only after
; determining Capital / Small
    mov char, al

    ; ///////////
    mov ah, 02h
    mov dl, 10
    int 21h
    mov ah, 02h
    mov dl, 13
    int 21h
    ; ///////////

    ; OUTPUT

    mov dx, offset msg2
    mov ah, 09h
    int 21h
```

```
    ; IF BLOCK (if Capital, make
Small)

    mov al, char
    storing `char` value to AL to compare
    with 61h
    cmp al, 60h
    comparing with the character before
    'a' (" " " ")
    jbe to_SMALL

    ; ELSE BLOCK (if Small, make
Capital)

    sub al, 20h

    jmp to_Output

to_SMALL:
    add al, 20h

to_Output:

    mov char, al

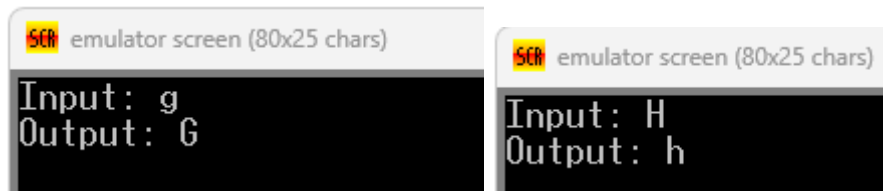
    mov ah, 02h
    mov dl, char
    int 21h

    exit:
    mov ah, 4ch
    int 21h

    main endp

end main
```

Output:



Discussions:

The relevant notes have been documented at:

[Microprocessor_Assembly_Lab/doc/Assignments/Learning/A_4-theory.md](https://github.com/bltranger/Microprocessor_Assembly_Lab/blob/main/doc/Assignments/Learning/A_4-theory.md) at main · bltranger/Microprocessor_Assembly_Lab

Also see: (From which I learned that we are working with **flags** instead of actual value changes (**If-Else** checks)).

Master Jump Summary Table

Mnemonic	Alternate Name	Meaning / Condition Checked	Flag Evaluation	Data Type
JMP	—	Unconditional Jump	None (Always jumps)	Any
JE	JZ	Jump if Equal / Zero ($==$)	$ZF = 1$	Any
JNE	JNZ	Jump if Not Equal / Not Zero ($\neq$)	$ZF = 0$	Any
JA	JNBE	Jump if Above ($>$)	$CF = 0 \wedge ZF = 0$	Unsigned / ASCII
JAE	JNB , JNC	Jump if Above or Equal ($\geq$)	$CF = 0$	Unsigned / ASCII
JB	JNAE , JC	Jump if Below ($<$)	$CF = 1$	Unsigned / ASCII
JBE	JNA	Jump if Below or Equal ($\leq$)	$CF = 1 \vee ZF = 1$	Unsigned / ASCII
JG	JNLE	Jump if Greater ($>$)	$ZF = 0 \wedge SF = OF$	Signed Integers
JGE	JNL	Jump if Greater or Equal ($\geq$)	$SF = OF$	Signed Integers
JL	JNGE	Jump if Less ($<$)	$SF \neq OF$	Signed Integers
JLE	JNG	Jump if Less or Equal ($\leq$)	$ZF = 1 \vee SF \neq OF$	Signed Integers
JS	—	Jump if Sign / Negative	$SF = 1$	Signed
JNS	—	Jump if No Sign / Positive	$SF = 0$	Signed
JO	—	Jump if Overflow	$OF = 1$	Signed Overflow
JNO	—	Jump if No Overflow	$OF = 0$	Signed Safe
JP	JPE	Jump if Parity Even	$PF = 1$	Bit parity check
JNP	JPO	Jump if Parity Odd	$PF = 0$	Bit parity check
JCXZ	—	Jump if CX is Zero	$CX = 0$	Loop bounds check

Conclusion:

I have successfully completed the assignment. While working on it, I had to study the **Procedure Calling** and **Level Jumping** to decide which one to use for this assignment.